

Appendix: Artifact Description

PDSW'26 — WarpX / ADIOS2 I/O benchmarking study

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 Balanced-workload write-path characterization on Frontier and Perlmutter (weak scaling; multi-TB/s aggregate throughput, including Blosc2 effective bandwidth).
- C_2 Comparison of ADIOS2 aggregation strategies (TwoLevelShm, EveryoneWritesSerial, DataSizeBased) under extreme writer-side skew.
- C_3 Evaluation of FlattenSteps for assembling lab-frame BTB snapshots from non-contiguous boosted-frame contributions at negligible write-time overhead.
- C_4 Evaluation of SPAN and AsyncWrite for buffering and compute/I/O overlap under BTB imbalance.
- C_5 Practical ADIOS2 configuration guidelines for WarpX-like PIC codes.

B. Computational Artifacts

- A_1 <https://github.com/guj/artifacts26>
Git repository containing Frontier build/env scripts, WarpX inputs, PerformanceRun and repeat/ job scripts, log timer extractor, and result CSVs corresponding to this paper. (Optional Zenodo DOI may be added after acceptance for badge applications.)

Artifact	Contrib.	Related paper elements
A_1	C_1 – C_5	Table 1; Figs. BTB-a (FlattenSteps), BTB-b (SPAN/Async), BTB-c/d (aggregation); small- N mean±stdev (§II-B, §4); §6 recommendations

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 packages the experiment inputs, ADIOS2 parameterizations, repeat/ scripts for small- N three-trial runs, the log timer extractor, and numerical results used to produce the paper's throughput tables and BTB figures. It supports independent replotting from CSVs (Google Sheets) and (where allocations permit) re-execution of the same WarpX → openPMD-api → ADIOS2 write path.

Expected Results

Reproducing A_1 should recover:

- The Frontier 3D Cartesian throughput table (EWS/TLS ± Blosc2) to within normal shared-filesystem variation; relative ordering of configurations is the primary claim.
- BTB relative rankings: FlattenSteps overhead negligible vs. EWS; SPAN+AsyncWrite highest among tested BTB configs; DataSizeBased preferred when subfile count is constrained under skew.

Absolute TB/s on production Lustre is not expected to be bit-identical across days.

Expected Reproduction Time (in Minutes)

- Setup: 30–120 min (build WarpX/ADIOS2 stack or load site modules).
- Small-scale smoke test (1–8 nodes): on the order of tens of minutes per configuration (site-dependent).
- Analysis / replot from CSVs: <15 min.
- Full paper scale (up to 2048 Frontier / 512 Perlmutter nodes): requires leadership-class allocations; not expected for routine AE.

Artifact Setup (incl. Inputs)

Hardware:

- Frontier (ORNL): HPE Cray EX; AMD MI250X GPUs; Lustre/ClusterStor ORION (≈5 TB/s peak write). Weak scaling reported to 2048 nodes; 8 CPU cores/node dedicated to I/O in the paper. Primary documented platform: setup and job scripts in the artifact repository target Frontier.
- Perlmutter (NERSC): GPU nodes; all-flash Lustre scratch (>5 TB/s aggregate). Weak scaling reported to 512 nodes (scratch-quota limited); 4 CPU cores/node for I/O. Uses the same WarpX → openPMD-api → ADIOS2 stack and job pattern as Frontier; only site modules, account/QOS, and scratch paths differ. Example Slurm scripts are in `warp_x_tests/PerformanceRun/` (e.g. BTB/N8/perlmutter.sh); adapt as needed for other scales.

Node-local NVMe burst buffers were explored for BTB staging but are not required to regenerate the primary CSV/plot artifacts.

Software:

- WarpX 25.12 (https://github.com/BLAST-WarpX/warp_x)
- openPMD-api (external install; linked into WarpX with `WarpX_openpmd_internal=OFF`)

- ADIOS2 2.11, BPFile engine / BP5 (<https://github.com/ornladios/ADIOS2>)
- Frontier modules used by scripts/frontier/bash.setup.env: cmake/3.30.5, craype-accel-amd-gfx90a, rocm, cray-mpich/8.1.31, cce/18.0.1, hdf5/1.14.5-mpi (optional ccache, ninja)
- Lossless Blosc2 via ADIOS2 operator (zstd level 1, bit-shuffle, 2048-byte threshold, 6 threads/rank) as in the paper table caption
- Python 3.6+ for scripts/frontier/extract_time_3.6.py (TotalTime / IOTime from job logs)
- Google Sheets (or equivalent) to import results/*.csv and export paper figures — no matplotlib drivers are shipped

Datasets / Inputs: No external observational datasets. WarpX decks and primary Slurm scripts are in warpx_tests/; small- N three-trial scripts are in repeat/:

- inputs_3d/opmd/regular/, regular_blosc/, regular_h5/: balanced 3D Cartesian LWFA weak scaling ($\approx 33\text{M}$ cells/node; field-dominated $\approx 2.5\text{ GB/node/step}$).
- inputs_3d/opmd/btd_*/, btd_flatten*/: boosted-frame BTM (skew + out-of-order arrival; FlattenSteps variants).
- PerformanceRun/3D/, PerformanceRun/BTD/: Frontier job scripts (plus BTM/.../perlmutter.sh).
- repeat/: same job pattern, regrouped by config (bp/, bp_async/, bp_flatten/, h5/); submit each script three times for the paper's $N=8\text{--}32$ repeats (see repeat/README.md).
- PerformanceRun/SST_BTMD/: exploratory staging (secondary).

Per-scale decks use input.n\${NNODES}. Scripts under repeat/ resolve EXE, bpls, and inputs_3d relative to warpx_tests/ (set #SBATCH -A YOUR_PROJECT). Aggregated numbers used for the paper plots/tables are in results/:

- frontier_3d_throughput.csv — Table 1 (TB/s)
- fig_btd_a_flatten_frontier.csv — Fig. BTM-a (GB/s)
- fig_btd_b_span_async_frontier.csv — Fig. BTM-b (GB/s)
- fig_btd_c_agg_perlmutter.csv — Fig. BTM-c (GB/s)
- fig_btd_d_subfiles_perlmutter.csv — Fig. BTM-d (GB/s)
- repeat_runs_raw.csv, repeat_summary.csv — three-trial TotalTime/IOTime (mean $\pm$ stdev)

See warpx_tests_INDEX.md, repeat/README.md, and results/README.md in the artifact repository.

Installation and Deployment: Frontier is the documented build/run host. Choose a scratch working directory WORKDIR (no hard-coded user path) and run:

- 1) export WORKDIR=/lustre/orion/<project>/scratch/<user>/<exp>

- 2) source scripts/frontier/bash.setup.env (loads modules; sets ADIOS2_DIR, openPMD_DIR, AMREX_AMD_ARCH=gfx90a, GPU-aware MPI).
- 3) ./scripts/frontier/build.setup builds and installs ADIOS2 $\rightarrow$ openPMD-api $\rightarrow$ WarpX under \$WORKDIR/binary/ (override tags with ADIOS2_TAG, OPENPMD_TAG, WARPX_TAG).
- 4) In warpx_tests/, symlink EXE $\rightarrow$ WarpX binary and bpls $\rightarrow$ ADIOS2 bpls; set #SBATCH -A to your allocation.
- 5) Submit, e.g. sbatch PerformanceRun/3D/N1/frontier_blosc.sh or sbatch PerformanceRun/BTD/N8/frontier.sh. For small- N repeats: from repeat/..., submit each script three times (same inputs/pattern as PerformanceRun/).
- 6) From the parent of each job_id/ directory, run python3 scripts/frontier/extract_time_3.6.py job_id (bare job id, no trailing slash) to obtain TotalTime and IOTime. Option 1 I/O time is TotalTime_{I/O} – TotalTime_{nullcore} when nullcore runs are available; DataSize is the aggregate on-disk output file size observed from each run.
- 7) Aggregate times and sizes into results/*.csv; replot by importing CSVs into Google Sheets and exporting charts.

Perlmutter uses the same stack and inputs_3d/; start from the example scripts under PerformanceRun/ (e.g. BTM/N8/perlmutter.sh) and adjust account/QOS, modules, and scratch paths.

Artifact Execution

Workflow: T_1 build/load stack $\rightarrow T_2$ run WarpX configs $\rightarrow T_3$ extract_time_3.6.py plus observed file sizes into CSV $\rightarrow T_4$ Google Sheets tables/figures. Parameters of interest: node count (weak scaling), aggregation method, NumSubFiles, FlattenSteps on/off, SPAN on/off, AsyncWrite on/off, Blosc2 on/off. Selected Frontier configs at $N=8\text{--}32$ were repeated three times (repeat/); larger scales and Blosc remain single-trial, with reliability argued via cross-scale consistency (paper §3).

Artifact Analysis (incl. Outputs)

- CSV tables of aggregate write throughput (TB/s) vs. node count, plus small- N repeat summaries (mean $\pm$ stdev).
- Plots corresponding to BTM FlattenSteps, SPAN/AsyncWrite, and aggregation studies (recreated from CSVs in Google Sheets).
- Comparison against paper Table 1 and BTM figures: match relative trends; document filesystem contention / re-runs as in the paper Limitations.

Note: Artifact Evaluation (AE) appendix intentionally omitted for submission. Authors may add AE after acceptance if applying for badges.